

Создаем собственный Wasaby-модуль

Шаг 1: Установка wasaby-cli

Установите wasaby-cli по инструкции из [статьи](#).

Шаг 2: Создание интерфейсного модуля

Создать модуль можно с помощью wasaby-cli, выполнив команду [createModule](#):

```
1 npx wasaby-cli createModule --path=client/MyModule
```

При ошибке выполнения команды укажите атрибуты `--kaizen`, `--responsible` и `--responsibleId`, например:

```
1 npx wasaby-cli createModule --path=client/MyModule --kaizen="289f760b-74ce-403f-ae72-2d204b4b69d8" --responsible="Копнин И.И." --responsibleId="3b06dd00-1a9d-4a62-bede-73a4639b97d7"
```

Шаг 3: Создание контрола

Подробную документацию с описанием базового класса контрола Wasaby вы можете прочитать в [соответствующем разделе](#).

Скопировать пример простейшего контрола можно из [папки ControlExample на github](#).

Обратите внимание на то, что основной файл с исходным кодом контрола на TypeScript расположен в папке `_process` (имя вы можете дать любое, но папка должна называться с подчеркивания (`"_"`)).

TypeScript отвечает не только за реализацию исходного кода, но и за подключение зависимостей от wml-шаблонов и css-файлов.

```
1 // TypeScript
2 import 'css!ControlExample/process';
3
4 export default function Diagram() {
5   return <div>Hello, world!</div>;
6 };
```

Обратите внимание, что в корне модуля публикуется TypeScript-файл для объявления библиотеки файлов (в примере этого модуля файл `process.ts`).

```
1 // TypeScript
2 export {default as Diagram} from './_process/Diagram';
```

Зависимость именно от этого файла будут подключать потребители вашего контрола. Во время сборки в файл библиотеки будут запакованы все зависимые исходники.

При обращении из `wml`:

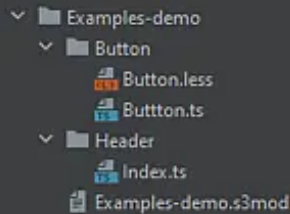
```
1 <!-- WML -->
2 <ControlExample.process:Diagram/>
```

Шаг 4: Создание демо-примера

Если ваш контрол не описывает страницу веб-приложения и предназначен для использования в других модулях, то для проверки и тестирования своего контрола вы можете создать дополнительный [интерфейсный модуль](#) с демо-примерами. Имя такого модуля должно заканчиваться на "-demo", например "Examples-demo".

У демо-модуля должна быть прописана зависимость от модуля, для которого описываются демо-примеры. Например, [Controls-demo](#) зависит от Controls.

Внутри модуля создайте демо-примеры. Каждый пример — это контрол, создание и конфигурация которого подчиняются [общим правилам](#). Имя TS-файла контрола должно либо совпадать с именем папки, в которой он расположен, либо соответствовать имени `Index.ts`.



 Исходные файлы контролов должны размещаться только в отдельных папках.

Подробнее о запуске демо-стенда читайте [здесь](#).

Шаг 5: Создание прямого маршрута

Если в вашем интерфейсном модуле нужно обеспечить доступ к контролу по прямой ссылке, то опишите роутинг. Для этого в корне модуля создайте файл `Index.tsx`:

```
1 // TypeScript
2 import { Diagram } from 'ControlExample/process';
3 import Bootstrap from 'UI/Bootstrap';
4
5 export default function Index(props) {
6     return (<Bootstrap>
7         <Diagram {...props} />
8     </Bootstrap>);
9 };
```

Для доступа к контролу по произвольному маршруту необходимо определить карту маршрутов в корне модуля в файле `router.json` в формате "путь: имя модуля".

```
1 // router.json
2 {
3     "/index.html" : "ControlExample"
4 }
```